

Regime maps for exact quantum compilation separate expressivity, certificates and feature transfer

Sze Chun Yiu

Quantum compilation is instance dependent, but a performance footprint alone does not distinguish an intrinsic compiler property from a proof limitation or a failed representation. We define an exact regime map that records reference-compiler exactness, constructive improvement mechanisms, intrinsic support, proof-validity regions, feature determination and prospective transfer as separate objects. Four frozen models show why the distinctions matter. A shared-tag Pauli block-encoding model has an all-size support-two ceiling, whereas a rank-two dependent-triple model has intrinsic support number one and a nontrivial objective region where the support-one proof applies. A six-term linear-combination model admits an exact structural boundary. Weighted Clifford state preparation behaves differently. Its complete $n \leq 3$ domain contains 1,146 states, and a sign-aware 127-feature representation separates all labels into 1,109 cells. This representation is nearly injective rather than compact, and it does not transfer: parity-split lookup makes 32 errors on a frozen 120-state $n = 4$ panel, covers only two states, and is indistinguishable from a shuffle null ($p = 0.51$). An earlier prospective forecast also failed, matching 100 of 120 regime labels and 67 of 120 exact costs. These results do not supply a universal compiler law or a physical advantage claim. They show that exact compiler studies should report what is intrinsic, what is certified, what is representation dependent and what survives prospective falsification.

Keywords: quantum compilation; exact optimization; stabilizer circuits; block encoding; al-

gorithm selection; instance-space analysis.

1 Introduction

Selecting an algorithm from instance features is a classical problem [1]. Modern instance-space analysis turns features and performance into interpretable regions that reveal where algorithms succeed or fail [2, 3]. The same perspective has reached quantum optimization and compilation. It has been used to study parameter initialization [4], quantum versus classical algorithm choice [5], compilation-option prediction [6], and device-specific compiler selection [7]. These studies establish that feature-conditioned performance mapping and algorithm selection are prior work, not contributions of this paper.

Exact compiler models permit a different question. When an exact optimum or a complete proof is available, one can ask why a reference compiler fails, whether a restricted family always contains an optimum, and whether the feature representation itself determines the label. These questions have different logical answers. A support bound may be intrinsic or merely sufficient for one proof. A proof can hold only for a region of objective weights. A feature map can fit a complete finite domain yet fail on new sizes. Combining these answers into one accuracy surface hides the distinction.

We therefore study a *regime map* as a typed record rather than a single phase diagram. The record separates six objects: the reference-exact region, constructive improvement mechanisms, intrinsic support, proof-derived support ceilings, objective-conditioned proof validity, and feature determination with prospective transfer. A negative result is part of the map. For example, mixed feature cells prove that no classifier using those features can be exact on the stated domain, while a failed prospective forecast shows that an

Sze Chun Yiu: sze-chun.yiu@fysik.su.se

in-domain rule did not transfer.

The evidence comes from four frozen quantum-compilation models (Table 1). Two Pauli block-encoding models admit all-size support results. A six-term linear-combination model admits an exact incumbent boundary. A weighted Clifford stabilizer-state model supplies complete finite graphs and a prospective size-transfer test. The models are deliberately heterogeneous. They are examples that separate logical coordinates, not a statistical sample of compilers.

The central result is that the coordinates do separate in practice. In one model, an all-size support-two theorem coexists with a still-refinable vocabulary of named mechanisms. In another, a safe syndrome-rank ceiling of five coexists with intrinsic support number one. A proof-validity cone does not imply the opposite theorem outside the cone. A near-injective state representation can determine every in-domain label and still fail out of sample. These distinctions constrain how exact compiler results should be reported and compared.

2 Regime-map definitions and study design

2.1 Exactness and support

Let $x \in \mathcal{X}$ be an instance, let θ specify objective weights, and let $\mathcal{C}(x)$ be the feasible compiler configurations. The exact cost is

$$C^*(x; \theta) = \min_{c \in \mathcal{C}(x)} C(c; x, \theta). \quad (1)$$

A deterministic reference compiler returns cost $C_D(x; \theta)$. Its exact region is

$$\mathcal{R}_D(\theta) = \{x : C_D(x; \theta) = C^*(x; \theta)\}. \quad (2)$$

Equality is always indexed by the model, domain and objective. A finite census, an all-size theorem and a frozen panel are reported as different evidence classes.

For models with a support coordinate $s(c)$, define the intrinsic support number by

$$\kappa(\theta) = \min\{k : \forall x \exists c^* \text{ with } s(c^*) \leq k\}. \quad (3)$$

A proof of sufficiency gives an upper bound on κ . Calling the bound intrinsic also requires tightness, such as infeasibility below it.

2.2 Constructive mechanisms and proof-validity regions

When the reference is inexact, a mechanism consists of a feasible witness, a strict cost reduction, and the structural edit that produces it. We use mechanism names only after these three parts are checked. A growing mechanism vocabulary does not weaken a separate theorem about the full feasible family.

A normalization proof may depend on θ . Its *proof-validity region* is the set of objective weights for which every local replacement is non-increasing and the analytic composition remains valid. Outside that set, the certificate is silent. A counterexample or a necessity theorem must be established separately.

2.3 Feature determination

Let $\phi(x)$ be a frozen feature vector and $y(x) = 1[x \in \mathcal{R}_D]$. A feature cell is the set of instances sharing one vector. A cell is mixed if it contains both labels. The minimum error attainable by any deterministic classifier using only ϕ on a finite domain is

$$E_{\min}(\phi) = \sum_v \min\{n_0(v), n_1(v)\}, \quad (4)$$

where $n_b(v)$ counts label b in cell v . Thus, $E_{\min} > 0$ is a representation-level impossibility result. Conversely, $E_{\min} = 0$ only proves determination on the registered domain. Cell count and singleton count reveal whether that determination is compressive or nearly memorizing.

2.4 Model register

The shared-tag Pauli model contains three paired blocks. Each block uses anticommuting Pauli frames and all blocks share one binary tag relation. The objective charges raw frame support, tag support and restoration support, with central frame letters cheaper than noncentral letters. The support coordinate is the maximum frame-Pauli weight.

The rank-two dependent-triple model contains two blocks. Two anticommuting generators determine a third frame in each block, and the blocks share a two-bit tag. Its unit objective charges commuting and noncommuting frame resources, tag support and restoration support.

Table 1: The four compiler models answer different questions about exactness. “All sizes” means all sizes admitted by the stated algebraic model, not arbitrary compiler or hardware settings. Finite entries state the complete or frozen domain used.

Model	Exact authority	Headline result	Boundary that remains
Shared-tag Pauli block encoding	all sizes	exact optima need frame support at most 2	the smallest named support-two trade vocabulary is still refinable
Rank-two dependent-triple block encoding	all sizes	intrinsic support number $\kappa = 1$ under the unit objective	the support-one proof has an objective-dependent validity region
Six-term linear-combination compilation	all admitted batches; complete checks at $n = 1, 2$	the reference compiler is exact exactly at the proved pair-gain boundary	transfer beyond the frozen cost model is not claimed
Weighted Clifford stabilizer-state preparation	complete graphs for $n \leq 3$; frozen 120-state panel at $n = 4$	exact failures split into order, pivot, route and global mechanisms	an in-domain feature representation does not transfer to the frozen $n = 4$ panel

Table 2: Feature determination and transfer for weighted Clifford state preparation. The 127-feature representation determines the complete $n \leq 3$ labels but is nearly injective and fails the pre-specified transfer criterion.

Quantity	Complete $n \leq 3$	Frozen $n = 4$ panel
States	1,146	120
Feature cells	1,109	120
Singleton cells	1,072	118
Mixed cells	0	0 in-panel
Irreducible in-domain errors	0	0 in-panel
Parity-split lookup errors	–	32
Covered states in parity split	–	2
Shuffle-null mean errors	–	32.41
Empirical p (errors no larger)	–	0.51

The support coordinate is the maximum support of an independent generator.

The six-term linear-combination model compares a reference factoring construction with an unrestricted family under a frozen structural cost combining selection, preparation and width terms. Its proof reduces all admitted partition shapes to inequalities in pair-derived gains.

The state-preparation model uses $H, S, S^\dagger$ and controlled-NOT gates with costs 1, 1, 1, 3. The

reference is a deterministic Gaussian-elimination-style stabilizer synthesis. Exact costs are shortest-path distances from $|0 \cdots 0\rangle$ in the complete stabilizer-state graph. Complete graph sizes are 6, 60, 1,080 and 36,720 for $n = 1, 2, 3, 4$, respectively. Complete analyses use $n \leq 3$; the $n = 4$ study uses a frozen 120-state panel.

2.5 Verification and prospective discipline

All-size results combine written compositions with complete finite checks of the local obligations. Complete finite-domain results enumerate every admitted state or instance. Selected generators were rerun from a clean working directory, and a separate standard-library verifier checked immutable input digests, stated fields, arithmetic and manuscript bindings without importing the generators. These checks remain within one project and one custody chain. They are not external replication.

For the state-preparation transfer test, the rule and panel keys were fixed before exact $n = 4$ labels were opened. Both confirmations and failures were retained. The later 127-feature study fixed its feature construction and in-domain criterion before the $n = 4$ evaluation, then evaluated transfer without changing the criterion.

3 Results

3.1 Shared-tag Pauli block encoding has a support-two ceiling

Theorem 1 (Support-two normalization). *For every size and target admitted by the shared-tag Pauli model under its unit raw-support objective, an exact optimum exists in which every frame Pauli has support at most two.*

The proof considers a frame R of support at least three, its anticommuting partner, and the shared tag. Each active qubit receives a two-bit class: its local commutation contribution with the partner and its local tag contribution. The partner relation makes the first class-bit parity odd. Every multiset of at least three such classes contains either a zero class or a repeated class. Removing the corresponding one or two letters preserves both parities, so feasibility is retained without modifying the tag.

The cost change is local. A complete 18,432-case check shows that restoration cost can never exceed the refunded frame cost. Repeated removal therefore reaches support at most two without increasing cost. A written parity argument covers arbitrary support; enumeration of 43,688 class tuples corroborates the support-two boundary and the support-three-and-higher descent.

The theorem closes the support envelope, not the smallest explanatory vocabulary inside it. Exact counterexamples successively require split anchors, support borrowing, off-support borrowing and hybrid constructions. These mechanisms remain within support two. Thus, intrinsic expressivity can be closed while a compact human-readable taxonomy remains refinable.

3.2 The rank-two model has intrinsic support one

Theorem 2 (Tight support-one normalization). *For every size admitted by the rank-two dependent-triple model under the unit objective, an exact optimum exists with generator support at most one. Support zero is infeasible, so $\kappa = 1$.*

The decisive transformation localizes each dependent-triple block to one anticommuting core and then reconstructs the shared tag. Preserving the old tag during every local edit had

produced looser bounds. Reconstruction exposes deletion credits that pay for core alignment and, when needed, relocation of the tag.

The local obligations are complete. They comprise 2,880 deletion cases, 6,912 core-alignment cases, 576 same-core tag-rigidity cases and 9,216 distinct-core tag lower-bound cases. The largest core-alignment penalty is three, while a deleted active column earns at least four. Distinct cores require tag cost eight; equal cores require cost four. These inequalities close every case in the all-size composition.

A conserved-syndrome abstraction gives a safe ceiling of five for this model. The tight theorem gives one. The difference shows that syndrome rank measures what a particular deletion proof preserves, not necessarily what an optimal compiler intrinsically needs. On the two models where both quantities and a local exchange margin are available, rank minus intrinsic support equals the measured margin. This is a two-point, rewrite-dependent observation, not a law.

3.3 The support-one proof has an objective region

Let $t_c, t_{nc}, t_{\text{tag}}, t_r$ weight commuting frames, non-commuting frames, tag support and restoration support. The same support-one proof remains valid when

$$2t_{nc} \geq 5t_r, \quad t_c + t_{nc} \geq 5t_r, \quad (5)$$

$$2t_{nc} \geq 2t_r + 2t_{\text{tag}}, \quad t_c + t_{nc} \geq 2t_r + 2t_{\text{tag}}. \quad (6)$$

When $t_c \leq t_{nc}$, the two inequalities involving $t_c + t_{nc}$ suffice for this certificate. The unit objective lies on the boundary, and a registered interior point passes. Several outside controls correctly fall outside. At any point outside, the certificate is silent. It does not say that support two is necessary.

A corrected search over 211,248 frozen-domain candidates found zero strict support-two witnesses under four objectives. This strengthens the domain-bounded negative result but does not prove global sharpness of the cone.

3.4 The six-term model has an exact structural boundary

Theorem 3 (Reference-exact boundary). *For every admitted six-term batch and size in the*

frozen linear-combination model, the reference construction is exact if and only if the proved pair-gain condition holds.

The proof enumerates 203 partitions across 11 shapes and reduces each shape to exact affine block-gain inequalities. Converse witnesses show that a violated clause produces a cheaper admitted construction. A separate complete regression checks 729 instances at $n = 1$ and 38,760 at $n = 2$ with zero mismatches. On the $n = 2$ domain, one pair-gain literal represents the boundary exactly. This compact boundary is a property of the declared model and objective, not evidence that other compiler boundaries are low order.

3.5 State preparation transfers mechanisms but not feature generalization

Exact shortest-path censuses on the complete $n \leq 3$ graphs identify 5, 28 and 189 reference-exact states out of 6, 60 and 1,080. Strict improvements fall into four mechanism classes. They arise from changing processing order, pivot choice, elimination route, or the global synthesis family. The last class shows that no tested row-reduction schedule closes the complete domain.

The prospectively frozen $n = 4$ forecast failed. It matched 100 of 120 regime labels and 67 of 120 exact costs. The mismatches were retained rather than used to redesign the rule. This result rejects the proposed transfer criterion on the panel.

A coarse 13-feature representation provides a stronger in-domain negative. Twelve feature cells are mixed, which forces at least 43 errors among the 1,146 complete $n \leq 3$ states for every classifier using only those features. Adding sign-aware state structure changes the in-domain result. The 127-feature representation has no mixed cells and no irreducible in-domain error. It produces 1,109 distinct cells, including 1,072 singleton cells. This is finite-domain vocabulary existence, not a compact law.

The richer representation does not repair transfer (Table 2). In a parity-split lookup test on the frozen $n = 4$ panel, only two states are covered and the remaining labels produce 32 of 120 errors. The shuffle-null mean is 32.41 errors, and the empirical probability of an error count no larger than observed is $p = 0.51$. The pre-specified transfer criterion therefore fails.

4 Discussion

The four models support a conservative synthesis. Exactness regions are useful, but they do not have one authority or one explanation. A theorem about the full feasible family can coexist with an incomplete named taxonomy. A proof-derived search ceiling can be much larger than intrinsic support. An objective region can describe where a proof works without describing what is necessary outside. A feature vocabulary can determine a complete finite domain while carrying almost one cell per instance and failing at the next size.

This separation clarifies the relation to algorithm selection. A performance footprint answers which method wins on which features. The present record asks additional questions when exact structure is available. Is there a constructive witness for the gap? Does every optimum admit a normal form? Is the normal form tight? Which objective weights preserve the proof? Does the representation determine the label? The answers can constrain or explain a footprint, but they do not replace instance-space analysis.

The state-preparation result gives the strongest warning against overcompression. Eliminating mixed cells by adding sign-aware features might appear to solve the boundary problem. The cell counts show otherwise. Most complete-domain states become singletons, and transfer coverage collapses to two panel states. The conversion is scientifically useful because it localizes the earlier failure to missing sign information. It does not produce a reusable predictor.

The same logic applies to support. The syndrome-rank ceiling remains useful for restricting search, even though it is not tight. The tight support-one result required a richer proof move, namely global tag reconstruction. Proof-language improvement, rather than larger enumeration alone, exposed the intrinsic value.

For practitioners, the proposed reporting order is direct. State the exact model and objective. Record where the reference is exact. Attach constructive witnesses to failures. Separate intrinsic support from a safe proof ceiling. Index every certificate by objective weights. Test whether features determine the label before increasing classifier capacity. Freeze transfer tests when an exact referee is available. This order keeps favorable and adverse results on the same

evidential footing.

5 Related work and contribution boundary

Algorithm selection and instance-space analysis already own the general idea of mapping feature-conditioned performance regions [1–3]. Quantum studies already select between quantum and classical algorithms [5], use instance spaces for quantum optimization [4], and predict compiler or device choices [6, 7]. We do not claim novelty for any of these tasks.

Exact, verified and algebraic circuit optimization is also established. Prior work supplies verified optimizer transformations [8], exact pattern matching [9], graph-theoretic simplification [10], and optimal Clifford synthesis on bounded qubit counts [11]. Benchmark suites characterize compiler behavior across circuits and devices [12, 13]. These works absorb any claim that exact optimization, formal checking, compiler benchmarking or Clifford synthesis is introduced here.

The Pauli models build on established unitary partitioning and Pauli compilation [14–17]. Connectivity overhead and routing constraints have also been characterized independently [18]. The present structural costs do not replace hardware-aware resource models.

The surviving contribution is a typed combination. It places constructive gap witnesses, intrinsic versus proof-derived support, objective-conditioned proof validity, representation-level impossibility and prospective falsification in one exact-compiler record. Each component can succeed or fail independently. The paper does not introduce a new optimizer, benchmark or algorithm-selection method, and it does not claim that this combination is a universal theory of compiler regimes.

6 Limitations

The all-size theorems concern two exact algebraic grammars and frozen structural objectives. They do not establish physical runtime, fault-tolerant resource savings, hardware advantage or production-scale compiler behavior. The six-term result is likewise restricted to its admitted batch family and cost decomposition.

The state-preparation conclusions are finite. Complete state graphs are used only through $n = 3$, while transfer is tested on one frozen 120-state $n = 4$ panel. The 127-feature representation is sign aware and permutation covariant, but it is near-injective in domain and has negligible parity-split coverage on the panel. Other representations, gate sets, costs or measurement-assisted preparation may behave differently.

The objective inequalities describe a sufficient region for one support-one proof. They are not proved necessary. A corrected frozen-domain search found no strict outside witness, but a witness in a broader domain remains possible.

The rank-minus-support observation uses only two measured models and changes under a different rewrite alignment. It should be treated as a diagnostic coincidence until a third model discriminates between competing explanations.

Independent scripts and verifier implementations remain under one project and one custody chain. They check consistency and structural separation, but they are not independent external replication or external proof review. Public archival identity and third-party verification remain separate publication tasks.

7 Conclusion

Exact quantum-compilation studies can report more than a winner map, but the additional objects must remain distinct. Across four frozen models, an intrinsic support result separates from a named mechanism vocabulary, a proof ceiling separates from a tight normal form, a certificate region separates from necessity, and in-domain feature determination separates from transfer.

The decisive adverse result is the state-preparation test. A rich feature map removes all mixed cells on the complete $n \leq 3$ domain but is nearly injective and fails on the frozen $n = 4$ panel. The appropriate conclusion is not that every compiler has a simple regime geometry. It is that exact witnesses, proof scope, representation scope and prospective failures should be recorded separately before a compiler map is generalized.

8 Data and code availability

The review package contains the exact result records, table generator, selected replay

summaries and an independent standard-library binding verifier needed to check every headline number in this manuscript. The verifier does not import the scientific generators. It checks content digests, theorem fields, finite-domain arithmetic and the generated tables. Detailed proof obligations and adverse-result tables are provided in the Supplementary Information.

A permanent public archive identifier and reuse licence will be bound before filing. Until that deposition exists, no permanent identifier is asserted. The exact public archive and the submitted source must be linked without changing the audited scientific bytes.

Ethics, resources and author statement

This study used no human participants, animals, clinical records or personal data. Computation was used for exact enumeration, shortest-path optimization and finite proof obligations. Computational scale is a verification cost, not evidence of practical quantum advantage.

Sze Chun Yiu is the sole named author and is responsible for approving the scientific claims, analyses and final manuscript. Generative artificial-intelligence tools assisted with editorial refinement, literature-discovery support, code assistance and simulated review. All cited sources, numerical statements and final wording require author verification before submission.

Funding, competing-interest, acknowledgement, affiliation and identifier statements require final author confirmation before filing.

References

- [1] John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pages 65–118. Elsevier, 1976. DOI: [10.1016/S0065-2458\(08\)60520-3](https://doi.org/10.1016/S0065-2458(08)60520-3).
- [2] Kate A. Smith-Miles. Cross-disciplinary perspectives on meta-learning for algorithm selection. *ACM Computing Surveys*, 41(1): 1–25, 2009. DOI: [10.1145/1456650.1456656](https://doi.org/10.1145/1456650.1456656).
- [3] Kate Smith-Miles and Mario Andrés Muñoz. Instance space analysis for algorithm testing: Methodology and software tools. *ACM Computing Surveys*, 55(12):1–31, 2023. DOI: [10.1145/3572895](https://doi.org/10.1145/3572895).
- [4] Vivek Katial, Kate Smith-Miles, Charles Hill, and Lloyd Hollenberg. On the instance dependence of parameter initialization for the quantum approximate optimization algorithm: Insights via instance space analysis. *INFORMS Journal on Computing*, 37(1):146–171, 2025. DOI: [10.1287/ijoc.2024.0564](https://doi.org/10.1287/ijoc.2024.0564).
- [5] Charles Moussa, Henri Calandra, and Vedral Dunjko. To quantum or not to quantum: towards algorithm selection in near-term quantum optimization. *Quantum Science and Technology*, 5(4):044009, 2020. DOI: [10.1088/2058-9565/abb8e5](https://doi.org/10.1088/2058-9565/abb8e5).
- [6] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. Predicting good quantum circuit compilation options. In *2023 IEEE International Conference on Quantum Software*, pages 43–53, 2023. DOI: [10.1109/QSW59989.2023.00015](https://doi.org/10.1109/QSW59989.2023.00015).
- [7] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. MQT Predictor: Automatic device selection with device-specific circuit compilation for quantum computing. *ACM Transactions on Quantum Computing*, 6(1): 1–26, 2025. DOI: [10.1145/3673241](https://doi.org/10.1145/3673241).
- [8] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. A verified optimizer for quantum circuits. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–29, 2021. DOI: [10.1145/3434318](https://doi.org/10.1145/3434318).
- [9] Raban Iten, Romain Moyard, Tony Metger, David Sutter, and Stefan Woerner. Exact and practical pattern matching for quantum circuit optimization. *ACM Transactions on Quantum Computing*, 3(1):1–41, 2022. DOI: [10.1145/3498325](https://doi.org/10.1145/3498325).
- [10] Ross Duncan, Aleks Kissinger, Simon Perdrix, and John van de Wetering. Graph-theoretic simplification of quantum circuits with the ZX-calculus. *Quantum*, 4:279, 2020. DOI: [10.22331/q-2020-06-04-279](https://doi.org/10.22331/q-2020-06-04-279).
- [11] Sergey Bravyi, Joseph A. Latone, and Dmitri Maslov. 6-qubit optimal Clifford circuits. *npj Quantum Information*, 8:79, 2022. DOI: [10.1038/s41534-022-00583-7](https://doi.org/10.1038/s41534-022-00583-7).
- [12] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation. *ACM Trans-*

- actions on Quantum Computing*, 4(2):1–26, 2023. DOI: [10.1145/3550488](https://doi.org/10.1145/3550488).
- [13] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. MQT Bench: Benchmarking software and design automation tools for quantum computing. *Quantum*, 7:1062, 2023. DOI: [10.22331/q-2023-07-20-1062](https://doi.org/10.22331/q-2023-07-20-1062).
 - [14] Artur F. Izmaylov, Tzu-Ching Yen, Robert A. Lang, and Vladyslav Verteletskyi. Unitary partitioning approach to the measurement problem in the variational quantum eigensolver method. *Journal of Chemical Theory and Computation*, 16(1):190–195, 2020. DOI: [10.1021/acs.jctc.9b00791](https://doi.org/10.1021/acs.jctc.9b00791).
 - [15] Filipa C. R. Peres and Ernesto F. Galvão. Quantum circuit compilation and hybrid computation using pauli-based computation. *Quantum*, 7:1126, 2023. DOI: [10.22331/q-2023-10-03-1126](https://doi.org/10.22331/q-2023-10-03-1126).
 - [16] Ewout van den Berg and Kristan Temme. Circuit optimization of hamiltonian simulation by simultaneous diagonalization of pauli clusters. *Quantum*, 4:322, 2020. DOI: [10.22331/q-2020-09-12-322](https://doi.org/10.22331/q-2020-09-12-322).
 - [17] Jennifer Paykin, Albert T. Schmitz, Mohannad Ibrahim, Xin-Chuan Wu, and A. Y. Matsuura. PCOAST: A pauli-based quantum circuit optimization framework. In *2023 IEEE International Conference on Quantum Computing and Engineering*, pages 715–726, 2023. DOI: [10.1109/QCE57702.2023.00087](https://doi.org/10.1109/QCE57702.2023.00087).
 - [18] Pei Yuan and Shengyu Zhang. Full characterization of the depth overhead for quantum circuit compilation with arbitrary qubit connectivity constraint. *Quantum*, 9:1757, 2025. DOI: [10.22331/q-2025-05-28-1757](https://doi.org/10.22331/q-2025-05-28-1757).